



Vue Router 강의

SPA 라우팅의 핵심부터 가드까지 한 번에

라우트 매핑, 네비게이션, 동적 파라미터, 가드 실습으로 실전 감각을 익히는 완벽 가이드



전체 강의 구성

Vue Router 마스터를 위한 4단계 로드맵

01

싱글 페이지 앱(SPA) 라우터 구조 이해

Routes, RouterView, RouterLink의 역할과 동작 원리

02

라우트 등록 및 네비게이션 실습

RouterLink와 router.push()를 활용한 페이지 이동 구현

03

동적 라우팅과 파라미터 등록

URL 파라미터(:id) 처리와 useRoute() 혹 활용법

네비게이션 가드로 접근 제어하기

인증(Authentication) 미들웨어 구현 및 리다이렉션 처리

최종 결과물

기본 라우팅 프로젝트 완성

SPA 구조가 잡힌 Vue 3 애플리케이션

실전 패턴 적용

프로그래밍 방식 네비게이션 & 동적 파라미터 핸들링

보안 로직 구현

전역 가드를 활용한 로그인 접근 제어 시스템



모든 실습은 Vue 3 Composition API 기준으로
진행됩니다.

1

CHAPTER

싱글 페이지 앱(SPA) 라우터 구조 이해하기

SPA에서 주소만 바뀌고 화면을 컴포넌트로 교체하는 라우팅의 핵심 개념과 구조를 살펴봅니다.



ROUTES



VIEW



LINK

“SPA의 핵심은 새로고침 없는 화면 전환입니다.



페이지 로딩 방식의 변화

기존 웹사이트는 페이지 전체를 다시 로드하지만, SPA(Vue)는 주소만 변경하고 내부 컴포넌트만 교체하여 마치 페이지가 이동한 것처럼 보이게 합니다.



사용자 경험(UX) 향상

서버 렌더링 방식보다 훨씬 빠른 전환 속도를 제공하여, 사용자에게 앱과 같은 부드러운 경험을 제공합니다.

★ Vue Router 3대장

1

Routes

URL ↔ 컴포넌트 매핑 설정

2

RouterView

컴포넌트를 화면에 출력하는 영역

3

RouterLink

새로고침 없는 이동 링크 생성

"이 3가지만 알면 라우팅 기초 끝!"

Vue Router 핵심 개념 정리



Routes

CONFIGURATION

URL 경로와 컴포넌트를 연결하는 매핑 테이블입니다. 어떤 주소에 어떤 화면을 보여줄지 정의하는 설정의 중심입니다.



RouterView

OUTLET

현재 URL에 매칭된 컴포넌트가 렌더링되는 위치입니다. 주소가 바뀌면 이 영역의 내용만 교체됩니다.



RouterLink

NAVIGATION

HTML `<a>` 태그처럼 보이지만, 페이지 새로고침 없이 URL만 변경하여 부드러운 이동을 처리합니다.



SPA 라우팅의 이점

Why we use Vue Router?



빠른 화면 전환



부드러운 사용자 경험 (UX)



앱 같은 사용성



Vue Router 기본 디렉토리 구조

CLI로 생성한 기본 Vue 3 프로젝트에 라우터 설정을 위한 폴더와 파일들이 어떻게 구성되는지 확인해봅시다.



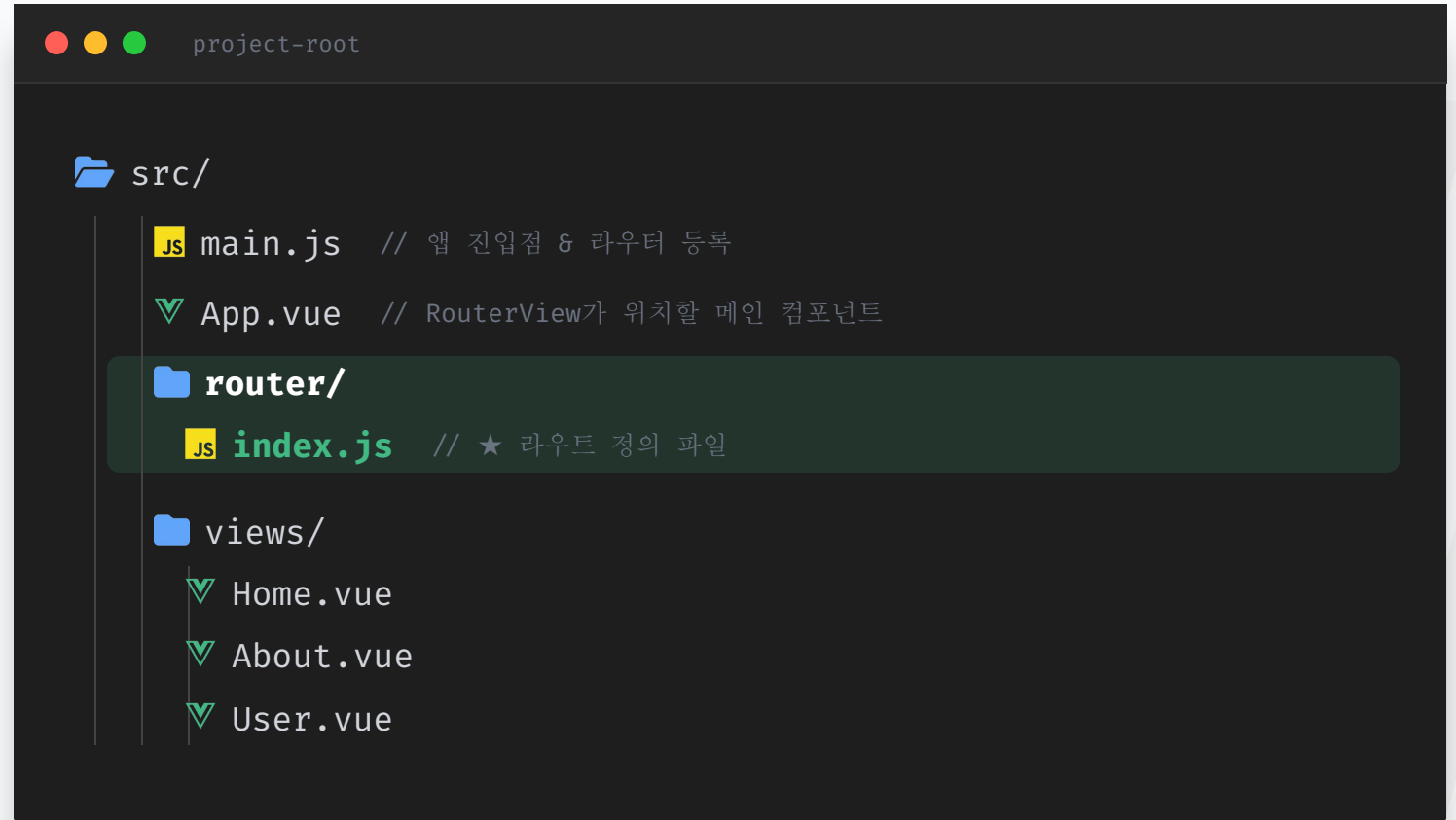
router/index.js

라우트 매핑 설정의 핵심 파일



views/

페이지 역할을 하는 컴포넌트들을 모아두는 폴더



```
npm install vue-router
```

Vue Router 설정의 핵심

URL과 컴포넌트를 매핑하고, 라우터 인스턴스를 생성하여 내보내는 전체 코드입니다.

- 1 라이브러리 & 컴포넌트 Import**
vue-router 함수들과 각 페이지 컴포넌트를 가져옵니다.
- 2 Routes 배열 정의**
path(URL)와 component(화면)를 객체 형태로 매핑합니다.
- 3 Router 인스턴스 생성**
WebHistory 모드를 설정하고 routes를 주입합니다.

```
src/router/index.js

1 import {createRouter, createWebHistory} from 'vue-router'
2 import Home from '../views/Home.vue'
3 import About from '../views/About.vue'
4 import User from '../views/User.vue'
5
6 const routes = [
7   {path: '/', component: Home},
8   {path: '/about', component: About},
9   {path: '/user/:id', component: User}, // 동적 라우트
10 ]
11
12 const router = createRouter({
13   history: createWebHistory(),
14   routes,
15 })
16
17 export default router
```



JS main.js

앱 생성 및 라우터 플러그인 등록

Entry Point

```
src/main.js

import { createApp } from 'vue'
import App from './App.vue'
import router from './router/index'

const app = createApp(App)
app.use(router) // 라우터 사용 등록
app.mount('#app')
```

Point:

app.use(router)를 호출해야 애플리케이션 전체에서 <RouterView>, <RouterLink> 등을 사용할 수 있습니다.

App.vue

네비게이션 및 라우터 뷰 배치

Root Component

```
src/App.vue

<template>
  <header>
    <RouterLink to="/">Home</RouterLink>
    <RouterLink to="/about">About</RouterLink>
  </header>
  <main>
    <!-- URL에 맞는 컴포넌트 출력 -->
    <RouterView />
  </main>
</template>
```

Point:

<RouterLink>는 새로고침 없는 a 태그 역할을 하며, <RouterView> 자리에 실제 페이지 컴포넌트가 렌더링됩니다.

2

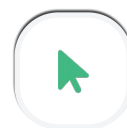
CHAPTER

라우트 등록 및 네비게이션 실습

라우트 정의와 RouterLink, 그리고 프로그래밍 방식 네비게이션 (router.push)을 실습합니다.



DEFINE



LINK



PUSH

이동은 간단하게, 제어는 강력하게



직관적인 라우트 등록

경로(path)와 컴포넌트(component)를 매핑하는 것만으로 등록이 완료됩니다. 복잡한 설정 없이 직관적인 구조를 가집니다.



자동 렌더링 시스템

RouterView는 현재 URL을 감시하고, 설정된 규칙에 따라 올바른 컴포넌트를 자동으로 화면에 갈아 끼워줍니다.

⇒ 페이지 이동의 2가지 방식



RouterLink

Template

HTML 태그처럼 사용하여 선언적으로 이동

```
<RouterLink to="/about">
```



router.push()

Script

로직 처리(로그인 등) 후 코드로 이동

```
router.push('/about')
```



스크립트에서 페이지 이동 제어하기

버튼 클릭이나 비동기 작업 완료 후 페이지를 이동시켜야 할 때는 router.push를 사용합니다.



useRouter Hook

Composition API에서는 useRouter를 통해 라우터 인스턴스에 접근합니다.



router.push()

히스토리 스택에 새 항목을 추가하여 뒤로 가기가 가능한 상태로 이동합니다.



팁: router.replace()를 사용하면 히스토리를 남기지 않고 현재 페이지를 교체합니다.

```
    <!-- 클릭 이벤트로 goAbout 함수 호출 -->
    <button @click="goAbout">About으로 가기</button>
  </div>
</template>

<script setup>
import { useRouter } from 'vue-router'

// 라우터 인스턴스 가져오기
const router = useRouter()

function goAbout() {
  // 프로그래밍 방식 네비게이션
  router.push('/about')
}
</script>
```

▼ About.vue

일반적인 컴포넌트 구조

View Component

views/About.vue

```
<template>
  <div>
    <h1>About 페이지</h1>
    <p>여기는 소개 페이지입니다.</p>
  </div>
</template>

<script setup>
  // 특별한 로직 없음
</script>
```

🔗 Template Navigation:

<RouterLink to="/about"> 태그를 사용하면 HTML에서 a 태그처럼 선언적으로 이동할 수 있습니다.



JS Programmatic

스크립트에서 이동 제어

Method Example

```
// 라우터 인스턴스 가져오기
const router = useRouter()

// 1. 히스토리에 추가 (일반 이동)
router.push('/about')

// 2. 히스토리 대체 (뒤로가기 불가)
router.replace('/about')

// 3. 뒤로 가기 (브라우저 Back)
router.back() // 또는 go(-1)
```

💡 Tip:

로그인 성공 후 리다이렉트처럼 로직 처리 후 페이지를 이동해야 할 때 router.push()를 주로 사용합니다.